

Neural Network Stochastic Differential Equation Models for Financial Data Forecasting



A project submitted
for partial fulfillment of the requirements for the award of the degree of
Master of Science

by
Ananda Biswas

M.Sc. in Statistics & Computing
Exam Roll No.: 24419STC053

Under the supervision of
Dr. Ganga Ram
DST-CIMS, BHU

Centre for Interdisciplinary Mathematical Sciences
I.Sc., Banaras Hindu University
Varanasi, India-221005

Declaration

I, **Ananda Biswas**, hereby declare that the project titled ***Neural Network Stochastic Differential Equation Models for Financial Data Forecasting*** and the work presented therein are my own and original, carried out under the supervision of **Dr. Ganga Ram, DST-CIMS**, Banaras Hindu University, during the period from March 2026 to May 2026. This project has been submitted to the DST-Centre for Interdisciplinary Mathematical Sciences, Banaras Hindu University, in partial fulfilment of the requirements for the M.Sc. in Statistics & Computing course. To the best of my knowledge and belief, the information and data presented in this project are authentic. This work has not been submitted to any other university or institution for the award of any degree, diploma, or fellowship, nor has it been published previously.

Place:

ANANDA BISWAS

Date:

24419STC053

CERTIFICATE

This is to certify that **Ananda Biswas**, a student of the **Master of Science in Statistics & Computing**, Semester IV of the **DST-Centre for Interdisciplinary Mathematical Sciences**, Institute of Science, Banaras Hindu University, bearing exam roll no. **24419STC053** and enrollment no. **482721** of 2024-2026 has successfully completed his project entitled “**Neural Network Stochastic Differential Equation Models for Financial Data Forecasting**”. The project was completed under my direct supervision and guidance for the partial fulfillment of the course curriculum of Master of Science in Statistics & Computing. He carried out the whole work with utmost sincerity, and the present report is the outcome of his sincere efforts and endeavors. This work either in the present form or in part in any other form is not being submitted anywhere to the best of my knowledge.

Supervisor

Course Coordinator

Coordinator, DST-CIMS

Acknowledgment

I am honoured to have this opportunity to convey my sincere gratitude to my supervisor, **Dr. Ganga Ram**, for his precious supervision, constant motivation, and caring mentorship throughout the tenure of this research. Without his immense knowledge and unparalleled vision, this thesis would not have taken this shape.

Place:

Date:

ANANDA BISWAS

24419STC053

*To my parents,
for every sacrifice they made*

ॐ

सरस्वति महाभागे विद्ये कमललोचने ।
विद्यारूपे विशालाक्षि विद्यां देहि नमोऽस्तुते ॥

Abstract

This study employs a collection of stochastic differential equations with drift and diffusion coefficients approximated by neural networks for financial time series forecasting. In particular, the performance of Stochastic Differential Equation Network with Brownian Motion and Lévy induced Stochastic Differential Equation Network with varying α are deployed on two major stock market indices : NIFTY50, India and SSE50, China. The models are employed to forecast index movement across multiple prediction horizons of 1, 3, 7, 14 days ahead, enabling both immediate and long-term prediction capabilities. The models aim to capture the stochastic and non-linear dynamics inherent in financial data through differential equation driven neural architectures. The findings suggest that stochastic differential equation based neural networks like SDENet, LDENet provide a promising approach for modelling highly volatile financial data with improved predictive performance.

Contents

Abstract	vi
List of Figures	viii
List of Tables	ix
1 Introduction	1
2 Methods	2
2.1 Deep Neural Networks	2
2.2 Residual Neural Networks	3
2.3 ResNet to Ordinary Differential Equations	4
2.4 Neural Ordinary Differential Equations	4
2.5 Neural Stochastic Differential Equations	5
2.5.1 Gaussian Noise	6
2.5.2 Non-Gaussian Noise	6
2.6 LDENet	8
3 Experiments	9
3.1 Dataset	9
3.2 Experiment 1 : Forecasting with Different Stock Patterns	11
3.3 Experiment 2 : Forecasting with Different Models	12
3.4 Experiment 3 : Forecasting with Different α	14
3.5 Experiment 4 : Effect of Forecasting Days on Prediction Accuracy	16
4 Conclusion	17
References	18

List of Figures

1	A Deep Neural Network	2
2	ResNet Structure	3
3	PDFs of Standard Gaussian and α -Stable Random Variables with different α, β	7
4	Brownian Motion and Lévy Motion with different α and β	8
5	Data set of NIFTY50 Index	9
6	Data set of SSE50 Index	9
7	Train-Test Split of NIFTY50	10
8	Train-Test Split of SSE50	10
9	NIFTY50-First Day Forecast Result	11
10	SSE50-First Day Forecast Result	11
11	NIFTY50-First Day Forecast Result for SDENet	12
12	NIFTY50-First Day Forecast Result for LDENet	12
13	SSE50-First Day Forecast Result for SDENet	13
14	SSE50-First Day Forecast Result for LDENet	13
15	SSE50-First Day Forecast Result for LDENet 1.2	14
16	SSE50-First Day Forecast Result for LDENet 1.5	14
17	SSE50-First Day Forecast Result for LDENet 1.8	14
18	SSE50-First Day Forecast Result for LDENet 1.99	15
19	Forecast Results for Different Prediction Horizons	16

List of Tables

1	Forecast Results for Different Stocks ($\alpha = 1.5$)	11
2	Forecast Results of Different Models	13
3	Forecast Results for Different α	15
4	Prediction Error over Forecasting Days	16

1 Introduction

Often, past information carried on by time series data can help one understand future phenomena. Time series forecasting is hereafter attractive in many research fields and lots of research work are developed to improve the prediction ability of the models. Traditionally, autoregressive models and moving average models are often applied to time series prediction. However, they essentially only capture linear relationship in the data set and can not afford many real cases, such as weather forecasting, financial market prediction or signal processing, which are often complex, nonlinear or even chaotic. Another class of statistical methods is Bayesian inference which is leveraged to deal with uncertainty in noisy environment and improve model accuracy by collecting some prior information. For example, Sahoo and Patra [1] combine the Bayesian inference method with Markov Chain Monte Carlo (MCMC) algorithm to evaluate water pollution rates. With the help of some prior knowledge, this method can greatly improve the prediction accuracy. However, if the prior distribution is not presumed correctly, the inference will go to the wrong direction. Hence, improving the reliability of prior knowledge is crucial to the model performance. As there always exists various kinds of uncertainties in the forecasting problems, how to accurately capture the intrinsic volatile nature of time series data is still challenging.

Recently, machine learning models outperform conventional methods in many aspects and have led a growing amount of researches to utilize various deep learning frameworks in the research of time series forecasting. Nowadays, the partnership between deep learning and dynamical systems has become a topic of increasingly popularity, as they could benefiting each other in both theoretical and computational aspects. On the computational side, for example, Neural ODE [2] is a combination of residual network and ordinary differential equations. For stochastic cases, neural stochastic differential equation (Neural SDE) [3] shows a continuous neural network framework based on stochastic differential equation with Brownian motions. The core of the SDE-Net is to treat the deep neural network transformation as the state evolution of a stochastic dynamic system, and introduce the Brownian motion term to capture cognitive uncertainty.

In many real world applications, Lévy noise plays an important part as a more general random fluctuation including complex non-Gaussian properties. Zhu [4] compares the Lévy noise with the standard Gaussian noise and concludes that the Lévy noise is more versatile and has a wider range of applications in diverse areas. The properties of Lévy motion enables it to better capture the uncertainty in financial data, such as abrupt transitions.

The rest of this report is organized as follows. In Section 2, Neural network models based on stochastic differential networks induced by Brownian motion and Lévy motions are presented. In Section 3, experimental results with real financial data are reported. Finally, I conclude the report and give future scopes of work in Section 4.

2 Methods

2.1 Deep Neural Networks

A Deep Neural Network (DNN) is a foundational machine learning architecture modelled on the human brain. Featuring multiple “hidden” layers of interconnected nodes, it processes complex, unstructured data like text, images, and audio to autonomously discover patterns, extract features, and make accurate predictions or decisions. [5] [6]

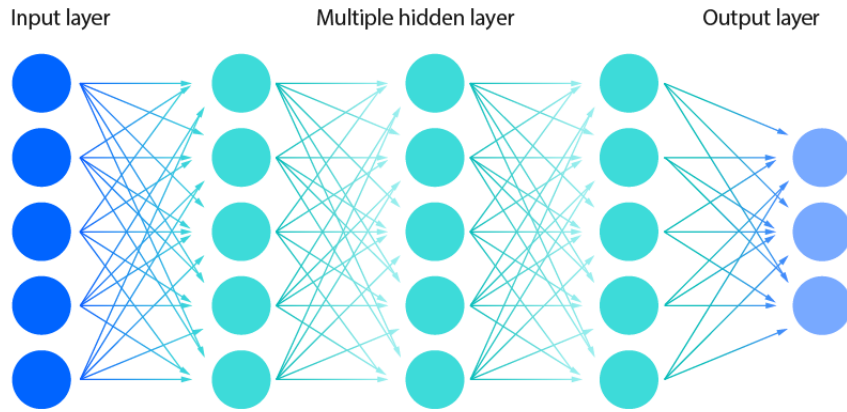


Figure 1: A Deep Neural Network

- DNNs are organized into three distinct types of layers, each playing a specific role in data transformation:
 1. **Input Layer:** Receives the raw data (e.g., individual pixels of an image or numerical values in a dataset) and passes it to the network.
 2. **Hidden Layers:** These are the multiple layers between the input and output layers. They perform nonlinear transformations to extract progressively complex features.
 3. **Output Layer:** Produces the final prediction, classification, or decision.
- The learning process in a DNN generally consists of the following steps:
 1. **Forward Propagation:** Data passes through the network where each connection applies weights and biases to compute activations.

2. **Loss Calculation:** The predicted output is compared with the actual target to compute the error or loss.
3. **Backpropagation:** The network updates its weights by propagating the error backward in order to minimize the loss function.

Some common DNN architectures are Convolutional Neural Networks, Recurrent Neural Networks etc.

2.2 Residual Neural Networks

As neural networks become deeper, they suffer from problems such as :

1. **Vanishing Gradient Problem :** During backpropagation, gradients can become extremely small, making earlier layers learn very slowly.
2. **Degradation Problem :** Adding more layers sometimes leads to worse training accuracy, even though deeper networks should theoretically perform better.
3. **Difficulty in Training Very Deep Networks :** Traditional deep networks become unstable and hard to optimize beyond a certain depth.

It often turns out that learning the residual is easier than learning the direct mapping. A Residual Neural Network (ResNet) [7] is a neural network architecture that introduces skip connections to learn residual mappings instead of directly learning complex transformations.

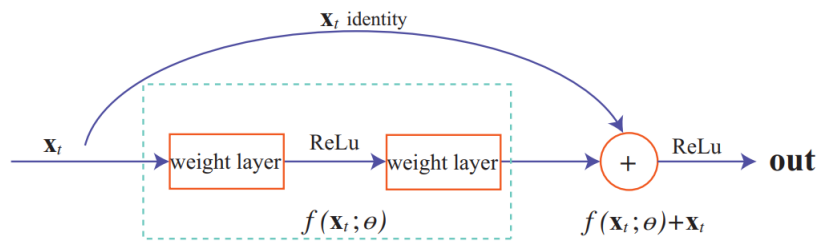


Figure 2: ResNet Structure

- The mapping of $f(\mathbf{x}_t; \boldsymbol{\theta})$ can be realised by feed-forward neural networks
- $\mathbf{x}_t$ is added to the output of the stacked layers
- The hidden state $\mathbf{x}_t$ of the t -th layer of the network can be described by the equation

$$\mathbf{x}_{t+1} = \mathbf{x}_t + f(\mathbf{x}_t; \boldsymbol{\theta}); \quad t = \{0, 1, 2, \dots, T\}. \quad (1)$$

2.3 ResNet to Ordinary Differential Equations

A growing amount of research arises in finding the relationship between neural network structure and discretized solution of numerical differential equations. For example, as neural network maps the input x to the output y through a series of hidden layers, the hidden representation can be regarded as the state of a dynamic system. In particular, ResNet can be regarded as a form of discretized Euler solution of ODEs.

Rewrite (1) as

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \boldsymbol{\theta}).$$

If we *add more layers* and *take small steps* i.e. $\Delta t \rightarrow 0$, then the continuous dynamics of hidden units can be shown as follows [2]

$$\begin{aligned} \frac{\mathbf{x}(t + \Delta t) - \mathbf{x}(t)}{\Delta t} &= f(\mathbf{x}(t); \boldsymbol{\theta}) \\ \Rightarrow \frac{d\mathbf{x}(t)}{dt} &= f(\mathbf{x}(t); \boldsymbol{\theta}). \end{aligned} \tag{2}$$

Thus we see that ResNet can indeed match the *Euler discretization* of ordinary differential equations.

2.4 Neural Ordinary Differential Equations

A Neural Ordinary Differential Equation is given by

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t); \boldsymbol{\theta}) \tag{3}$$

where $\mathbf{x}(t)$: hidden representation at time t ,

$f(\cdot; \boldsymbol{\theta})$: a neural network,

$\boldsymbol{\theta}$: parameters of the neural network f .

- **Forward Pass:** Given *initial state* $\mathbf{x}(0)$ and the differential equation, we compute

$$\mathbf{x}(T) = \mathbf{x}(0) + \int_0^T f(\mathbf{x}(t); \boldsymbol{\theta}) dt.$$

$f(\mathbf{x}(t); \boldsymbol{\theta})$ represents *how the state evolves at each instant*.

- **Backward Pass and Learning θ**

1. Starting with an initial guess of θ , we calculate $\mathbf{x}(T)$ using an ODE solver

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t f(\mathbf{x}(t); \theta).$$

This is a *Forward Pass* in practice.

2. Then compute the loss, calculate the derivatives w.r.t. the parameters. This is *Backward Pass*.
3. Update θ using the derivatives and repeat the process until convergence.

2.5 Neural Stochastic Differential Equations

- **Formalization**

A Neural Stochastic Differential Equation is given by

$$d\mathbf{x}(t) = f(\mathbf{x}(t); \theta) dt + g(\mathbf{x}(t); \phi) dW_t \quad (4)$$

where $\mathbf{x}(t)$: hidden representation at time t ,

$f(\mathbf{x}(t); \theta)$: *drift* coefficient approximated by a neural network,

dt : time increment,

$g(\mathbf{x}(t); \phi)$: *diffusion* coefficient approximated by a neural network,

dW_t : random noise increment.

- **Forward Pass**

Based on the *Euler-Maruyama scheme*, a forward simulation of a SDE with Gaussian noise is approximated by

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t); \theta) \Delta t + g(\mathbf{x}(t); \phi) \sqrt{\Delta t} \epsilon_t. \quad (5)$$

- Two runs with same initialization produce different paths.
- We simulate many trajectories and take the average.

- **Backward Pass**

1. Starting at an initial state $\mathbf{x}(0)$, with initial approximations of $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$, obtain $\mathbf{x}(T)$ by many small steps Δt .
2. Compute the loss and calculate the derivatives w.r.t. the parameters.
3. Update $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ using the derivatives and repeat until convergence.

2.5.1 Gaussian Noise

- Depending on the nature of the random noise, there can be different classes of SDEs.
- The most common random noise is *Gaussian Noise* under which

$$dW_t \sim \mathcal{N}(0, dt).$$

- In the forward simulation, we set

$$\epsilon_t \stackrel{iid}{\sim} \mathcal{N}(0, 1), \text{ consequently } \sqrt{\Delta t} \epsilon_t \stackrel{iid}{\sim} \mathcal{N}(0, \Delta t).$$

2.5.2 Non-Gaussian Noise

Gaussian noise is not always enough to capture big jumps. Hence we introduce Non-Gaussian noise. One such noise is α -Stable Lévy motion.

- **α -Stable Random Variables :**

- A random variable X is said to be α -Stable if its n *i.i.d.* copies have the property

$$\sum_{i=1}^n X_i \stackrel{d}{=} n^{1/\alpha} X + b_n$$

and denoted as

$$X \sim S_\alpha(\sigma, \beta, \gamma)$$

where σ : scale parameter, β : skewness, γ : location parameter.

- $\sigma Z + \gamma \stackrel{d}{=} X$ where $Z \sim S_\alpha(1, \beta, 0)$.
- $X \sim S_\alpha(1, 0, 0)$ is called a Standard α -Stable random variable.
- $\alpha \in (0, 2]$ controls how heavy tails are.
- $\alpha = 2 \rightarrow$ Gaussian.

- $\alpha = 1 \rightarrow$ Cauchy.
- Smaller $\alpha \rightarrow$ heavier tails (more extreme values).

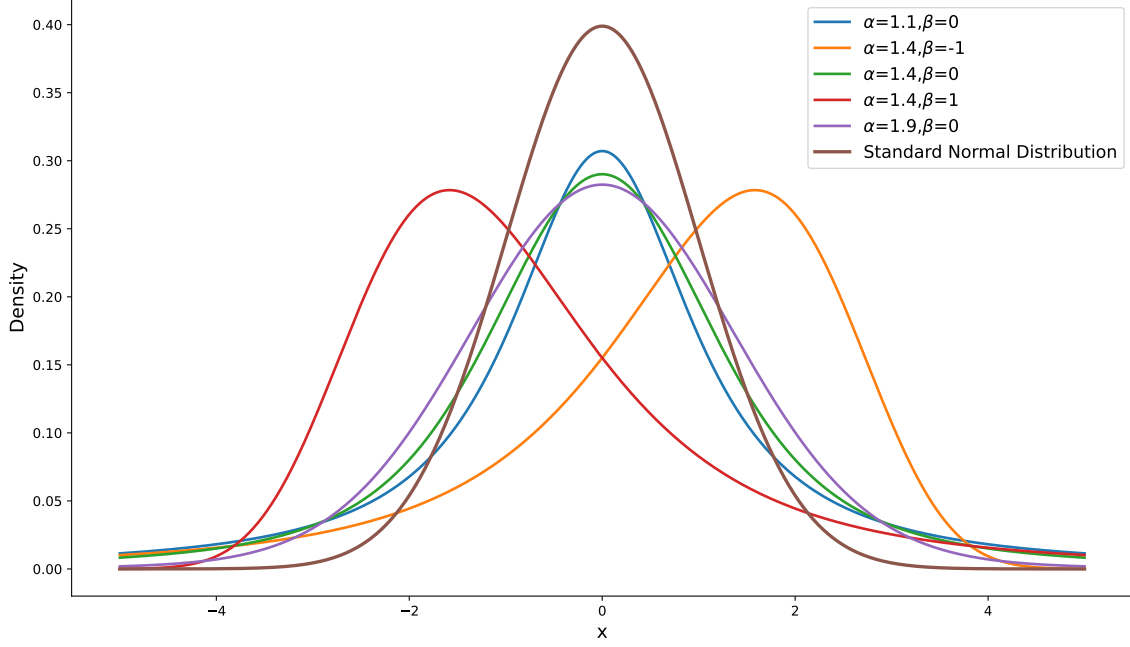


Figure 3: PDFs of Standard Gaussian and α -Stable Random Variables with different α, β

• α -Stable Lévy Motions

A symmetric α -Stable Lévy motion $\{L_t^\alpha\}_{t \geq 0}$ is a stochastic process with properties

- (i) $L_0^\alpha = 0$,
- (ii) L_t^α has independent increments,
- (iii) $L_t^\alpha - L_s^\alpha \sim S_\alpha \left((t-s)^{1/\alpha}, 0, 0 \right)$.

- For a very small step dt , $dL_{t+dt}^\alpha - dL_t^\alpha = dL_t^\alpha \sim S_\alpha \left((dt)^{1/\alpha}, 0, 0 \right)$.
- We can write $(dt)^{1/\alpha} L_t \stackrel{d}{=} dL_t^\alpha$ where $L_t \sim S_\alpha(1, 0, 0)$.
- In SDE, we set

$$L_t \stackrel{iid}{\sim} S_\alpha(1, 0, 0), \text{ consequently } (\Delta t)^{1/\alpha} L_t \stackrel{iid}{\sim} S_\alpha \left((\Delta t)^{1/\alpha}, 0, 0 \right).$$

- Euler-Maruyama approximation becomes

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t); \boldsymbol{\theta}) \Delta t + g(\mathbf{x}(t); \boldsymbol{\phi}) (\Delta t)^{1/\alpha} L_t.$$

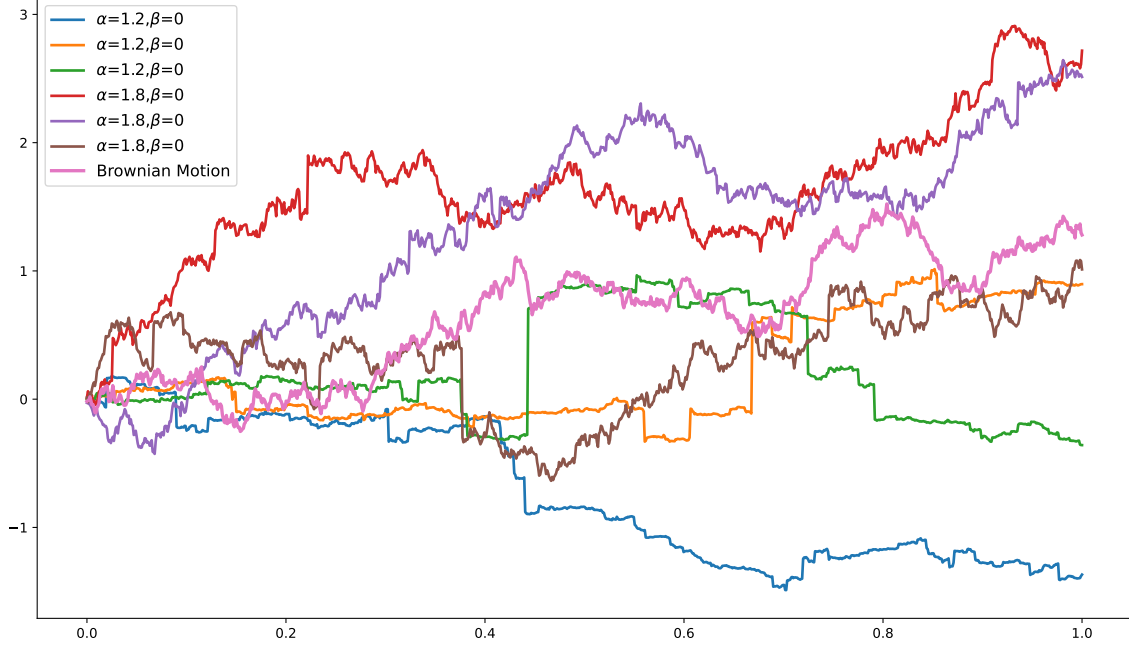


Figure 4: Brownian Motion and Lévy Motion with different α and β

2.6 LDENet

We add Lévy motion to the right side of equation (1) to make it the Euler discretization form of the stochastic differential equation. The continuous-time dynamics of the system (3) are then expressed as:

$$d\mathbf{x}(t) = f(\mathbf{x}(t); \boldsymbol{\theta}) dt + g(\mathbf{x}(t); \boldsymbol{\phi}) dL_t^\alpha, \mathbf{x}(0) = \mathbf{x}_0. \quad (6)$$

Based on the Euler-Maruyama scheme, the approximation for equation (6) is

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \underbrace{f(\mathbf{x}(t); \boldsymbol{\theta})}_{\text{drift neural net}} \Delta t + \underbrace{g(\mathbf{x}(t); \boldsymbol{\phi})}_{\text{diffusion neural net}} (\Delta t)^{1/\alpha} L_t. \quad (7)$$

Both drift and diffusion coefficients in equation (7) are estimated by two different multi-layer perceptron (MLP) neural networks which are called the drift net and diffusion net respectively. [8]

As of now, predictive performance is of interest. So we proceed with MSE Loss function.

3 Experiments

First we preprocess the real data through Phase Space Reconstruction with delay time $\tau = 1$ and embedding dimension = 30 and show the prediction accuracy for two different stock trending patterns. In addition, we also numerically analyze the effect of different α values in Lévy motion and forecasting days on prediction accuracy. The code of all experiments has been open sourced on GitHub : github.com/sakunisgithub.

3.1 Dataset

Due to the grave importance of financial market, and its relation to economic development, stock data are the best choice. Two datasets are selected corresponding to NIFTY50 Index of India and SSE50 Index of China — NIFTY50 Index with a total sample size of 4932 and SSE50 Index with a total sample size of 4917, both from Jan 2006 - Mar 2026.

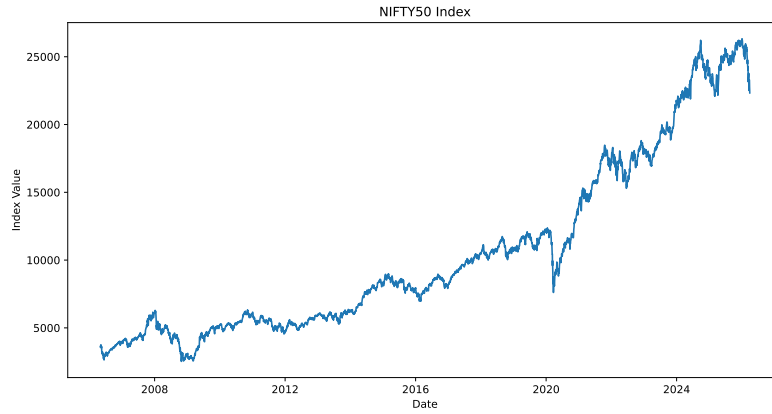


Figure 5: Data set of NIFTY50 Index

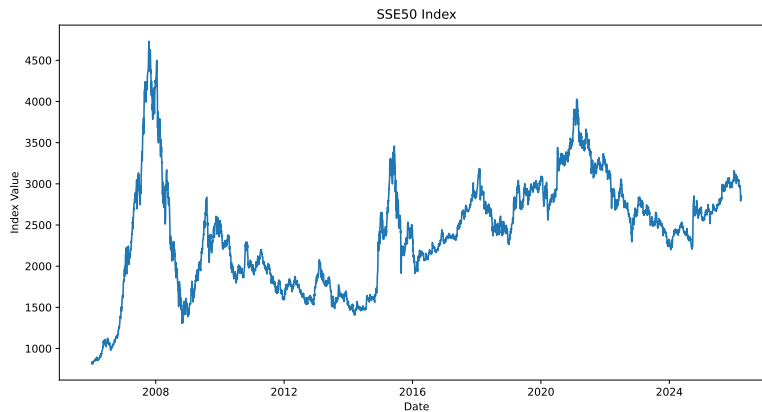


Figure 6: Data set of SSE50 Index

For both, data points from 2006 to 2020 were taken in training set and 2021 to 2026 data points were considered as test set.

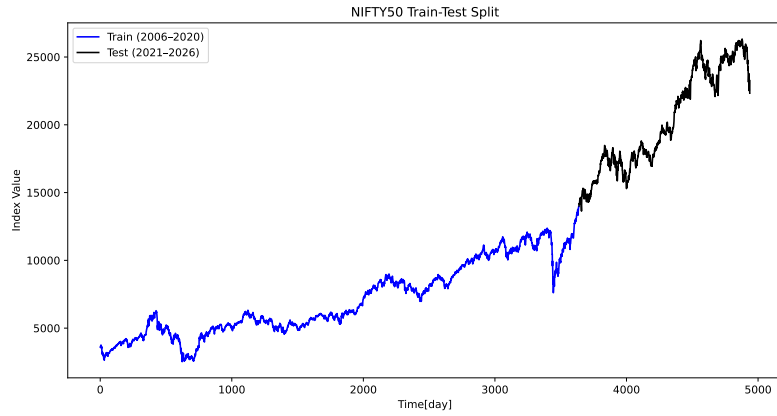


Figure 7: Train-Test Split of NIFTY50

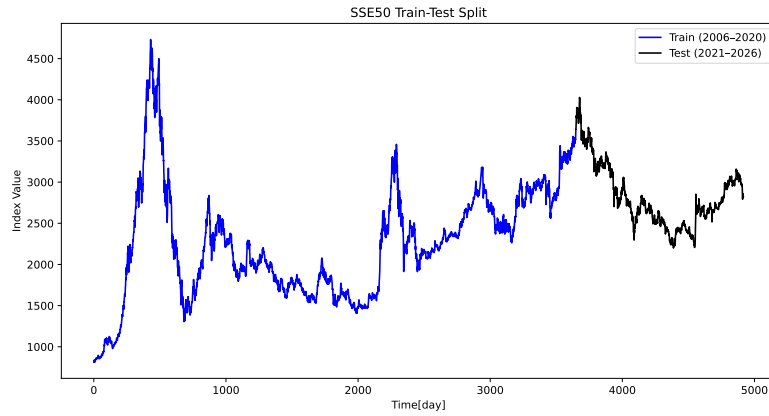


Figure 8: Train-Test Split of SSE50

For NIFTY50 Index, training set and test set fluctuations are small. For SSE50 Index, it shows large fluctuations on the training set and relatively small on the test set.

3.2 Experiment 1 : Forecasting with Different Stock Patterns

When comparing different stock trend types, we only consider the forecast effect of the first day under the same $\alpha = 1.5$. The results of loss can determine which type of stock data a model is more suitable for.

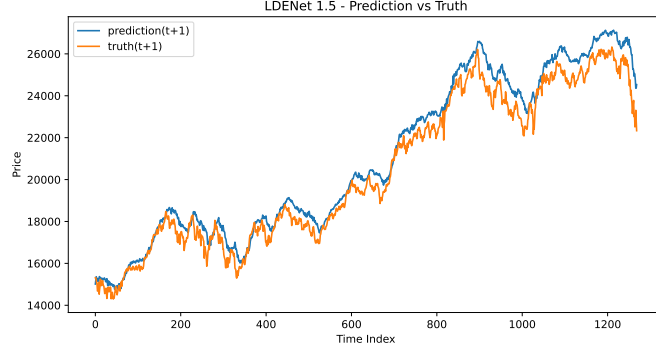


Figure 9: NIFTY50-First Day Forecast Result

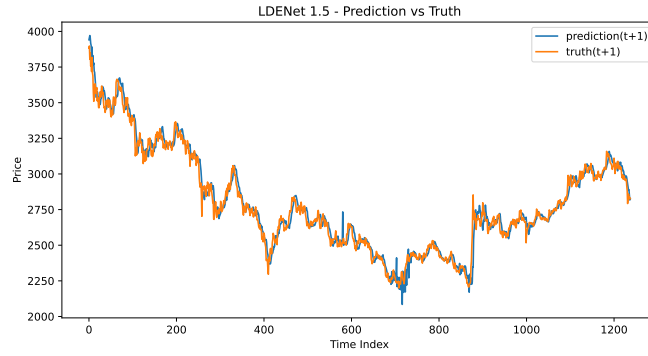


Figure 10: SSE50-First Day Forecast Result

Data	$MSE(t + 1)$	$MSE(t + 3)$	$MSE(t + 7)$	$MSE(t + 14)$
NIFTY50 Index	0.07797	0.01815	0.10413	0.16779
SSE50 Index	0.00718	0.00896	0.01834	0.02733

Table 1: Forecast Results for Different Stocks ($\alpha = 1.5$)

It can be seen from Figure (9), the prediction of NIFTY50 is poor, but that of SSE50 is praiseworthy. It helps us to comment that the LDE-Net model is suitable for predicting the future value of stocks with large fluctuations in historical data. When the training set fluctuates more frequently with larger amplitude, Lévy noise can better capture the uncertainty in the

prediction process, so as to achieve more accurate prediction. The historical data and test data of NIFTY50 did not have large fluctuations, hence Lévy noise could not give a good fit. In the follow-ups, we shall stick to SDE-Net for NIFTY50 and LDE-Net for SSE50 respectively.

3.3 Experiment 2 : Forecasting with Different Models

We now compare SDE-Net and LDE-Net for different scenarios. As prediction accuracy on first day is highest for either, it is reasonable to compare the effect of forecasting one day.

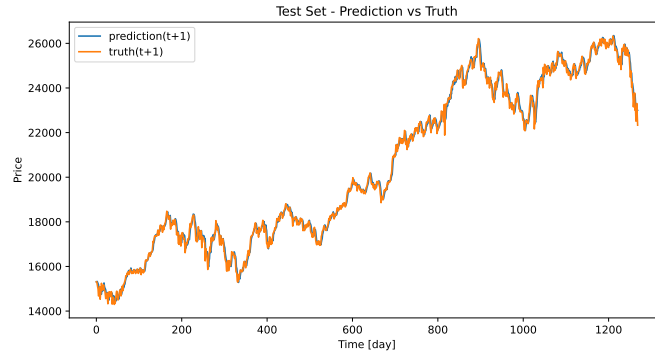


Figure 11: NIFTY50-First Day Forecast Result for SDENet

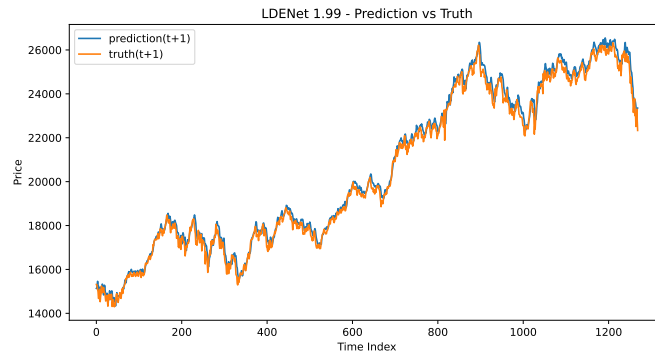


Figure 12: NIFTY50-First Day Forecast Result for LDENet

LDE-Net forecasts seem off as compared to SDE-Net forecasts of NIFTY50 Index values. Now let us have a look at the forecasts for SSE50 Index Values.

In case of SSE50, LDE-Net gives better forecasts as historical data have large fluctuations. These are supported by the following table as well.

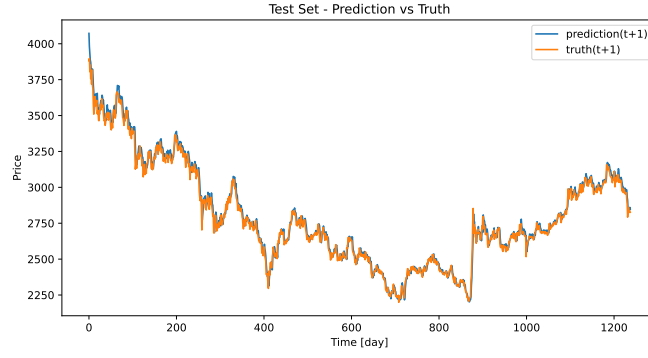


Figure 13: SSE50-First Day Forecast Result for SDENet

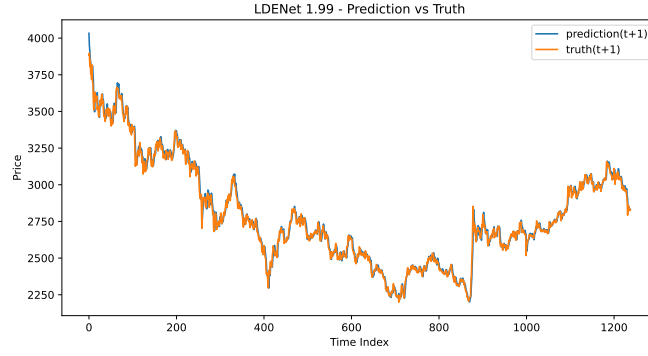


Figure 14: SSE50-First Day Forecast Result for LDENet

Data	Model	MSE($t + 1$)
NIFTY50 Index	SDE-Net	0.00598
	LDE-Net	0.01238
SSE50 Index	SDE-Net	0.00333
	LDE-Net	0.00271

Table 2: Forecast Results of Different Models

3.4 Experiment 3 : Forecasting with Different α

Consider that different α will produce different jumps, let us observe the effect on model prediction for $\alpha = 1.2, 1.5, 1.8, 1.99$.

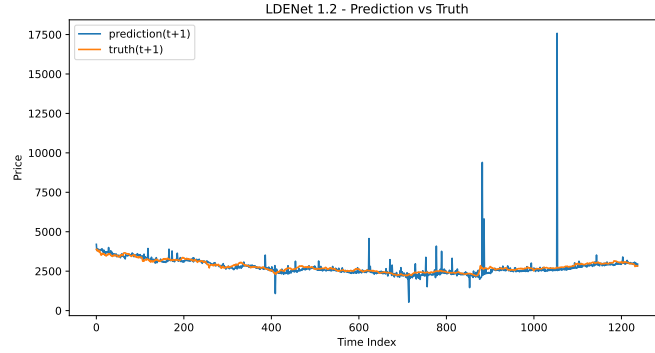


Figure 15: SSE50-First Day Forecast Result for LDENet 1.2

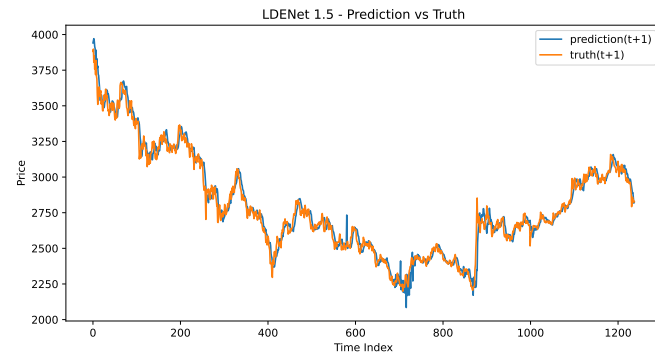


Figure 16: SSE50-First Day Forecast Result for LDENet 1.5

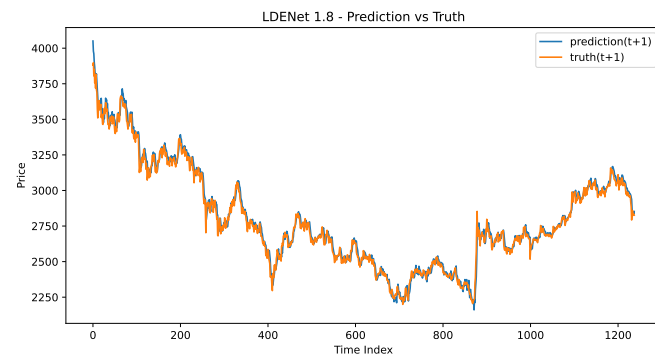


Figure 17: SSE50-First Day Forecast Result for LDENet 1.8

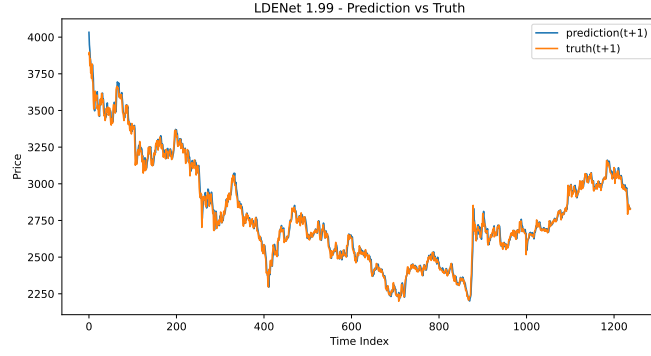


Figure 18: SSE50-First Day Forecast Result for LDENet 1.99

Data	Value of α	$MSE(t+1)$	$MSE(t+3)$	$MSE(t+7)$	$MSE(t+14)$
SSE50 Index	1.2	0.56702	0.61797	1.80748	1.90498
	1.5	0.00718	0.00896	0.01834	0.02733
	1.8	0.00362	0.00381	0.01481	0.02721
	1.99	0.00271	0.00302	0.01443	0.02509

Table 3: Forecast Results for Different α

From Table (3), it can be seen that the value of α has a certain effect on the prediction of the model. The greater the data fluctuation, the better a smaller α value; this directly corresponds to the fact that Lévy motion with smaller α produce greater jumps. For SSE50 Index value, LDE-Net with $\alpha = 1.99$ gives best predictive accuracy.

3.5 Experiment 4 : Effect of Forecasting Days on Prediction Accuracy

The accuracy over prediction days is also investigated and it is found that prediction error is an increasing function of prediction horizon, as the length of forecasting days increase, so does the prediction error. This behaviour is consistent throughout all choices of α as established in Table (3).

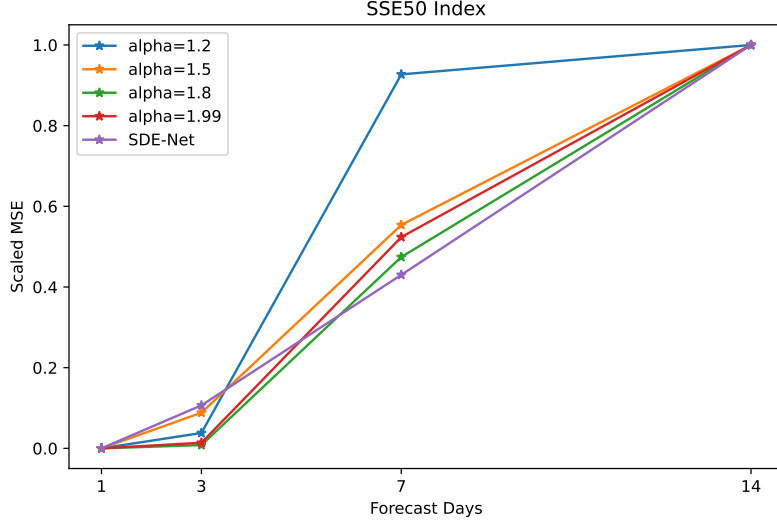


Figure 19: Forecast Results for Different Prediction Horizons

For NIFTY50, SDE-Net with Brownian Motion gives best predictive performance, while for SSE50, LDE-Net with $\alpha = 1.99$ performs the best in future predictions.

Data	Model	MSE($t + 1$)	MSE($t + 3$)	MSE($t + 7$)	MSE($t + 14$)
NIFTY50 Index	SDE-Net	0.00598	0.01635	0.03515	0.30617
	LDE-Net ($\alpha = 1.99$)	0.01238	0.01512	0.03962	0.30149
SSE50 Index	SDE-Net	0.00333	0.00646	0.01597	0.03272
	LDE-Net ($\alpha = 1.99$)	0.00271	0.00302	0.01443	0.02509

Table 4: Prediction Error over Forecasting Days

4 Conclusion

In this study, we work with a collection of stochastic differential neural networks equipped with Gaussian and Non-Gaussian noise. Brownian motion induced SDEs work fine for data-sets with smoothers fluctuations. On the other hand, special properties of α -stable Lévy motion induced SDEs help us to capture large fluctuations and uncertainty in a better way.

For different prediction horizons, predictive accuracy degrades whatever the model specification we use.

The model LDE-Net with suitable α is more suitable for predicting time series data that fluctuates with high volatility, as α -stable Lévy motion can better capture the training data trend with epistemic jumps.

This leaves us with plenty of future scope of work, a few might be optimal delay time, optimal choice of embedding dimension or a general procedure to obtain optimal α . Depending on the objective, different loss functions can also be employed for a better measurement of predictive error. Works can also be done to improve long-term predictive performance. Thus, stochastic differential equations equipped with neural networks propose a very promising framework for data with high volatility.

References

1. Sahoo, M. M. & Patra, K. C. River water quality modelling and simulation based on Markov Chain Monte Carlo computation and Bayesian inference model. *African Journal of Science, Technology, Innovation and Development* **12**, 771–785. <https://doi.org/10.1080/20421338.2019.1692460> (2020).
2. Chen, R. T. Q., Rubanova, Y., Bettencourt, J. & Duvenaud, D. *Neural Ordinary Differential Equations* 2019. arXiv: [1806.07366](https://arxiv.org/abs/1806.07366) [cs.LG]. <https://arxiv.org/abs/1806.07366>.
3. Liu, X. *et al.* *Neural SDE: Stabilizing Neural ODE Networks with Stochastic Noise* 2019. arXiv: [1906.02355](https://arxiv.org/abs/1906.02355) [cs.LG]. <https://arxiv.org/abs/1906.02355>.
4. Zhu, Q. Asymptotic stability in the pth moment for stochastic differential equations with Lévy noise. *Journal of Mathematical Analysis and Applications* **416**, 126–142. ISSN: 0022-247X. <https://www.sciencedirect.com/science/article/pii/S0022247X14001450> (2014).
5. LeCun, Y. & Hinton, G. Deep Learning. *Nature* **521**, 436–44. doi:[10.1038/nature14539](https://doi.org/10.1038/nature14539) (May 2015).
6. Goodfellow, I., Bengio, Y. & Courville, A. *Deep Learning* <http://www.deeplearningbook.org> (MIT Press, 2016).
7. He, K., Zhang, X., Ren, S. & Sun, J. *Deep Residual Learning for Image Recognition* 2015. arXiv: [1512.03385](https://arxiv.org/abs/1512.03385) [cs.CV]. <https://arxiv.org/abs/1512.03385>.
8. Yang, L., Gao, T., Lu, Y., Duan, J. & Liu, T. *Neural network stochastic differential equation models with applications to financial data forecasting* 2022. arXiv: [2111.13164](https://arxiv.org/abs/2111.13164) [cs.LG]. <https://arxiv.org/abs/2111.13164>.